

Python Core — 60 Minute Version

Condensed alternative to the full reference

Prep materials — Anthropic Fellows OA

August 2026

Contents

Day 1 — Python Core Refresher (60 min)	1
1. The type system in 5 minutes (8 min)	1
2. The four core containers (12 min)	2
3. Control flow and functions (10 min)	2
4. Comprehensions (8 min)	3
5. Sorting — the highest-value 10 minutes (10 min)	3
6. The three imports you'll need (7 min)	4
7. Errors, printing, and fast debugging (5 min)	4
Wrap-up (5 min)	4

Day 1 — Python Core Refresher (60 min)

For someone who codes well but hasn't written Python lately. Keep a REPL open (python3) and type every snippet. Skim what you already know; slow down on the boxes marked **GOTCHA** — those are where non-Python habits bite.

1. The type system in 5 minutes (8 min)

Dynamically typed, strongly typed: variables have no declared type, but values won't silently coerce.

```
x = 5          # int, arbitrary precision – no overflow, ever
y = 5 / 2      # 2.5  -> '/' ALWAYS returns float
z = 5 // 2     # 2    -> floor division
n = 5 % 2      # 1
p = 2 ** 10    # 1024 -> exponent
"a" + 1       # TypeError – no implicit coercion
```

GOTCHA — floor division rounds toward negative infinity: `-7 // 2 == -4` (not -3). This is why the ceiling-division idiom `-(-a // b)` works and why `int(a/b)` is a bug waiting to happen.

Type hints are optional and unenforced — decoration for readers:

```
def f(x: int, items: list[int]) -> str: ...
```

Truthiness: `0`, `0.0`, `""`, `[]`, `{}`, `set()`, `None` are all falsy. So if `items:` means “if non-empty”. `None` is the null; compare with `is None`, never `== None`.

2. The four core containers (12 min)

```
xs = [3, 1, 2]           # list – ordered, mutable
t = (3, 1)               # tuple – ordered, IMMUTABLE, hashable
d = {"a": 1, "b": 2}     # dict – insertion-ordered since 3.7
s = {3, 1, 2}           # set – unordered, unique, fast membership
```

Operations you'll actually use:

```
xs.append(4); xs.pop(); xs.pop(0)  # pop(0) is O(n) – use deque instead
xs.insert(0, 9); xs.remove(9)      # remove() deletes FIRST match by value
len(xs); 3 in xs                  # `in` on a list is O(n), on a set O(1)
xs[0]; xs[-1]; xs[1:3]; xs[::-1]  # slicing never errors on bad bounds
xs.sort()                         # in place, returns None
ys = sorted(xs)                   # returns a new list

d["a"]; d.get("z", 0)              # [] raises KeyError, .get() doesn't
d.setdefault("k", []).append(1)
for k, v in d.items(): ...
d.keys(); d.values(); "a" in d    # `in` on a dict checks KEYS
```

GOTCHA — assignment never copies:

```
a = [1, 2]; b = a; b.append(3)
a                # [1, 2, 3] – same object!
b = a[:]         # shallow copy
import copy; b = copy.deepcopy(a)
```

This is the single most common source of “impossible” bugs in a simulation where you reuse request objects across runs.

GOTCHA — never mutate a list while iterating it:

```
for r in running:           # WRONG – skips elements
    if r.done: running.remove(r)

running = [r for r in running if not r.done]  # right: rebuild
for r in list(running): ...                  # or iterate over a copy
running[:] = [...]                          # slice-assign mutates in place
```

3. Control flow and functions (10 min)

```
for i in range(5):          # 0..4
for i in range(2, 10, 3):   # 2, 5, 8
for i, x in enumerate(xs, start=1):
for a, b in zip(xs, ys):
while cond: ... break / continue

if a and not b or c: ...    # words, not && || !
val = a if cond else b      # ternary
```

Functions:

```
def f(a, b=10, *args, **kwargs):
    return a + b             # returns None implicitly if you omit `return`

def g(): return 1, 2         # returns a tuple
x, y = g()                  # unpacking
```

GOTCHA — mutable default arguments are created ONCE:

```
def bad(item, bucket=[]):      # NEVER do this
    bucket.append(item); return bucket
bad(1); bad(2)                 # [1, 2] — the same list both calls!

def good(item, bucket=None):
    bucket = [] if bucket is None else bucket
```

Lambdas are single-expression only — used almost exclusively as sort keys:

```
key = lambda r: (r.arrival, r.id)
```

4. Comprehensions (8 min)

Python's replacement for map/filter loops. Read them left-to-right after the first expression.

```
[x * 2 for x in xs]           # list
[x for x in xs if x > 0]      # with filter
[x if x > 0 else 0 for x in xs] # ternary goes BEFORE `for`
{k: v for k, v in pairs}      # dict
{x % 3 for x in xs}           # set
sum(r.tokens for r in reqs)    # generator — no brackets needed
```

Useful built-ins that pair with them:

```
sum(xs); min(xs); max(xs); len(xs); any(...); all(...)
sorted(xs, key=..., reverse=True)
```

5. Sorting — the highest-value 10 minutes (10 min)

Nearly every scoring bug in this kind of test is a tie-break bug.

```
xs.sort(key=lambda r: r.arrival)      # single key
xs.sort(key=lambda r: (r.arrival, r.id)) # tuple = multi-key
xs.sort(key=lambda r: (-r.priority, r.arrival)) # negate for DESCENDING
xs.sort(key=..., reverse=True)        # reverses ALL keys
```

Tuples compare element by element, so $(1, 5) < (1, 9) < (2, 0)$. That's the whole trick: build a tuple whose order matches the rule you were given, and negate any field that should be descending.

Selecting one element with the same discipline:

```
min(range(len(loads)), key=lambda i: (loads[i], i)) # least, ties -> low idx
min(items, key=lambda it: (it.priority, -it.id))    # ties -> HIGHEST id
```

GOTCHA: `sorted()` is stable — equal keys keep their original order. Handy, but don't rely on it when the spec names an explicit tie-break. Write it out.

6. The three imports you'll need (7 min)

```
from collections import deque, defaultdict, Counter
import heapq, math

q = deque([1, 2]); q.append(3); q.popleft(); q[0] # O(1) both ends
d = defaultdict(list); d[k].append(v)          # no KeyError
c = Counter("aabbbc"); c.most_common(2)         # [('b',3), ('a',2)]

h = []
heapq.heappush(h, (finish_time, req_id))        # min-heap of TUPLES
finish, rid = heapq.heappop(h)                  # smallest first
```

Push tuples onto heaps so ties break deterministically — and put the tie-break field *in the tuple*, because heapq compares the whole thing (and will crash comparing objects that aren't orderable).

`math.ceil(a / b)` works but goes through floats; `-(-a // b)` is exact.

7. Errors, printing, and fast debugging (5 min)

```
try:
    ...
except KeyError as e:
    ...
raise ValueError("message")
assert cond, "message"          # your fastest self-check inside a loop

print(f"t={t} running={[r.id for r in running]} reserved={reserved}")
```

f-strings are your debugger. In a tick-based simulation, printing one line per tick with the queue and running sets finds off-by-one errors faster than any other technique. Do it by default, delete before submitting.

Wrap-up (5 min)

Open `day1_drills.py` and do drills c1-c8 (the non-OOP ones). If any takes more than 3 minutes, reread that section above. Answers are in `day1_drills_answers.py`.

Tomorrow: classes, dataclasses, and reading someone else's OOP codebase — which is what the test actually hands you.